

ព្រះរាជាណាចក្រកម្ពុជា

ជាតិ សាសនា ព្រះមហាក្សត្រ



REPORT OF

OBJECT-ORIENTED PROGRAMMING (OOP)

Group:I3-AMS-D

Name of Student	ID of Student	Score
Tom Somary	e20240229	

Lecturer : Mr.LONG PAKRIGNA(course)

Lecturer: Mr.VANN VISAL(TP)

2026-2027

1. Introduction

Python is a high-level, general-purpose programming language known for its readable syntax. It supports object-oriented programming (OOP), which organizes code around classes and objects. A class describes the data and behavior that its objects can have. For example, a Student class can represent students with individual names, ages, and scores.

2. Python Basic Data Types

Python provides standard primitive and collection types to handle data efficiently. Primitive types include string (str), integer (int), floating-point (float), and Boolean (bool). Collection types allow storing multiple data items with distinct structural properties:

Lists : Ordered, mutable sequences that allow duplicate values.

Tuples : Ordered, immutable sequences used for read-only datasets.

Sets : Unordered collections of unique elements (duplicates automatically removed).

Dictionary: Key-value mappings providing efficient lookups.

Example:

```
student_names = ["Lacima", "Jady", "GMengz", "Mann Doss"]
student_ages = (19, 20, 21, 18)
student_gpas = {3.5, 2.0, 2.5, 4.0}
student_record = {
    "name": "Lacima",
    "age": 19,
    "gpa": 3.5
}
```

3. Variables and Attributes

1.Local Variables

A local variable is a variable created inside a function or method. It can normally be accessed only within that function or method.

Example:

```python
class Student:
  def introduce(self): # Local Variable: created inside method
      message= "My name is Jaehyun."
      print(message)

student1 = Student()
student1.introduce()
# Accessing 'message' here raises an error # print(message) # NameError: name 'message' is not defined
```

2 Instance Attributes

Instance attributes are variables that belong to a specific object. Each object can have its own values for these attributes. They are commonly defined using self inside the __init__() method.

Example:

```python
class Student:
  def __init__(self, name, grade):
      # Instance attributes (Unique per object)
      self.name = name
      self.grade = grade

# Creating two distinct instances (objects)
student1 = Student("Sokha", 10)
student2 = Student("Vicheka", 11)

print(student1.name, student1.grade) # Output: Sokha 10 print(student2.name, student2.grade) #
Output: Vicheka 11
```

3 Class Attributes

A class attribute is a variable defined directly inside a class but outside its methods. It is shared by instances that access the attribute through the class, unless an instance overrides it with an attribute of the same name.

Example:

```python
class Student:
    # Class attribute (Shared by ALL instances)
    school_name = "Institute of Technology of Cambodia"

student1 = Student()
student2 = Student()
print(student1.school_name) # Output: Institute of Tech... print(student2.school_name) # Output:
Institute of Tech...
 # Updating at class level updates it everywhere
Student.school_name = "ITC"
print(student1.school_name)
 # Output: ITC
```

4. Modifiers and Access Control — Part 1

Python does not enforce Java-style public, protected, and private access modifiers in the same way. A name without a leading underscore is public by convention. A single leading underscore (such as _balance) signals that a name

is intended for internal use. A double leading underscore (such as __balance) triggers name mangling, which helps avoid accidental name clashes but is not a security boundary.

Static and class methods

A static method uses @staticmethod and receives no automatic self or cls argument. A class method uses @classmethod and receives the class as cls, making it useful for working with class-level data or alternative constructors.

Example:

```python
class BankAccount:
    def __init__(self, owner, balance):
        self.owner = owner
        self.__balance = balance
        BankAccount.total_accounts += 1

    def get_balance(self):
        return self.__balance

    @classmethod
    def get_bank_name(cls):
        return cls.bank_name

    @classmethod
    def get_total_accounts(cls):
        return cls.total_accounts

    @classmethod
    def change_bank_name(cls, new_name):
        cls.bank_name = new_name

# Create two bank account objects
account1 = BankAccount("vathana", 500)
account2 = BankAccount("vayu", 1000)

# Use class methods
print("Bank:", BankAccount.get_bank_name())
print("Total accounts:", BankAccount.get_total_accounts())

# Access a private-style attribute through a method
print("vathana's balance:", account1.get_balance())

# Change a class-level attribute
BankAccount.change_bank_name("Smart Bank")
print("New bank name:", BankAccount.get_bank_name())
```

5. Modifiers and OOP Features — Part 2

An abstract class defines a shared interface or structure. In Python, the abc module and @abstractmethod can require subclasses to implement particular methods. An abstract class with unimplemented abstract methods cannot be instantiated.

An abstract classes is a class that provides a general structure for other classes. It cannot be instantiated directly when it contains methods that have not been implemented.

An abstract methods is a methods declared in an abstract class without providing its complete implementation

Encapsulation, properties, and final

Encapsulation groups data and behavior and provides controlled ways to use an object's internal state.

@property lets a method be accessed using attribute syntax, often for validation or computed values.

typing.final and @final mark a class as not intended for subclassing or a method as not intended for overriding; these annotations are primarily checked by static type checkers and are not general runtime enforcement.

Example:

```python
from abc import ABC, abstractmethod
import math
from typing import final

class Shape(ABC):
    """Abstract Base Class representing a geometric shape."""
    def __init__(self, color: str):
        self._color = color
    @property
    def color(self) -> str:
        return self._color
    @abstractmethod
    def area(self) -> float:
        pass
    @abstractmethod
    def perimeter(self) -> float:
        pass
@final
class Circle(Shape):
    """Concrete Final Class representing a Circle."""
    def __init__(self, color: str, radius: float):
        super().__init__(color)
        self.radius = radius
    @property
    def radius(self) -> float:
        return self.__radius
    @radius.setter
    def radius(self, value: float):
        if value <= 0:
            raise ValueError("Radius must be positive.")
        self.__radius = value
    def area(self) -> float:
        return math.pi * (self.__radius ** 2)
    def perimeter(self) -> float:
        return 2 * math.pi * self.__radius
class Rectangle(Shape):
    """Concrete Class representing a Rectangle."""
    def __init__(self, color: str, width: float, height: float):
        super().  init  (color)
```

```
        if width <= 0 or height <= 0:
            raise ValueError("Dimensions must be positive.")
        self.__width = width
        self.__height = height
    @property
    def dimensions(self) -> tuple[float, float]:
        return (self.__width, self.__height)
    def area(self) -> float:
        return self.__width * self.__height
    def perimeter(self) -> float:
        return 2 * (self.__width + self.__height)

try:
    shape = Shape("Red")
except TypeError as e:
    print(f"Instantiation Blocked: {e}")

circle = Circle(color="Blue", radius=5.0)
print("\nCircle Properties:")
print(f"Color: {circle.color} | Radius: {circle.radius}")
print(f"Area: {circle.area():.2f} | Perimeter: {circle.perimeter():.2f}")

try:
    circle.radius = -2
except ValueError as e:
    print(f"Validation Caught Error: {e}")

rect = Rectangle(color="Green", width=4.0, height=6.0)

print("\nRectangle Properties:")
print(f"Color: {rect.color} | Dimensions: {rect.dimensions}")
print(f"Area: {rect.area():.2f} | Perimeter: {rect.perimeter():.2f}")
```

6.Python Operators— Part 1

Operators perform calculations, comparisons, bit-level operations, and logical tests.

Category	Operators	Example
Arithmetic	+, -, *, /, //, %, **	10 // 3 == 3
Comparison	==, !=, >, <, >=, <=	80 >= 50 is True
Bitwise	&, |, ^, ~, <<, >>	5 & 3 == 1
Logical	and, or, not	score >= 50 and attendance >= 75

In student programs, comparison and logical operators can check whether a student passes, meets attendance requirements, or qualifies for an award.

Example:

```python
score = 80
attendance = 85

# 1. AND: Both conditions must be True
print(score >= 50 and attendance >= 75)
# Output: True

# 2. OR: At least one condition must be True
print(score >= 50 or attendance >= 90)
# Output: True

# 3. NOT: Reverses the Boolean result
print(not score >= 50)
```

7. Python Operators — Part 2

Assignment operators update variables, for example +=, -=, and *=. Membership operators in and not in test whether an item is present in a collection. Identity operators is and is not test whether two references point to the same object; they should not be used as substitutes for == when comparing values.

A conditional expression chooses one of two values: result = 'Pass' if score >= 50 else 'Fail'. Operator precedence determines the order of evaluation. For example, 2 + 3 * 4 evaluates to 14 because multiplication has higher precedence than addition. Parentheses can make the intended order clear.

Example:

```python
class Student:

    def __init__(self, name):

        self.name = name

        self.score = 50


s = Student("Alex")

s.score += 20

team = ["Alex", "Sam", "Mia"]

passed = "Yes" if s.score >= 60 else "No"

print(s.name in team, s.score * 2 + 10, passed)
```

8. Python Decision-Making Statements

The if statement runs code when a condition is true. if-else selects between two alternatives, while if-elif-else checks multiple conditions in sequence.

A nested if places one conditional inside another. Python 3.10 and later also support match-case for pattern-based selection, which can be useful for menu options.

Example:

testcode.py > ...

```python
students = {}

while True:
    print("\n=== Student Management System ===")
    print("1. Add student")
    print("2. View students")
    print("3. Search student")
    print("4. Exit")

    choice = input("Enter your choice: ")

    # Multiple-branch selection
    match choice:
        case "1":
            name = input("Enter student name: ")
            student_id = input("Enter student ID: ")

            # Nested if statements
            if student_id not in students:
                if name.strip():
                    students[student_id] = name
                    print("Student added successfully!")
                else:
                    print("Name cannot be empty.")
            else:
                print("Student ID already exists.")

        case "2":
            if students:
                print("\nStudent List:")
                for student_id, name in students.items():
                    print(student_id, "-", name)
            else:
                print("No students found.")

        case "3":
            student_id = input("Enter student ID: ")

            # Conditional expression
            result = (
                students[student_id]
                if student_id in students
                else "Student not found"
            )
            print(result)

        case "4":
            print("Goodbye!")
            break

        case _:
            print("Invalid choice. Try again.")
```

9. Python Loops and Loop Control

A for loop iterates over items in an iterable. A while loop repeats while its condition remains true. Python has no native do-while loop, but a while True loop with a break can imitate its execute-at-least-once behavior.

break terminates the nearest loop. continue skips the rest of the current iteration. pass does nothing and is used as a placeholder where a statement is syntactically required.



Loop control statements



Example:

```
class Student:
    def __init__(self, name, score):
        self.name = name
        self.score = score

students = [
    Student("Alex", 80),
    Student("Sam", 70),
    Student("Mia", 90),
]
```

Calculate the average

```
total = 0
for s in students:
    total += s.score
print("Average:", total / len(students))
```

```
> Average: 80.0
```

Search for a student

```
target = "Sam"
for s in students:
    if s.name == target:
        print("Found", s.name, s.score)
        break
```

```
> Found Sam 70
```

10. Java VS Python : Key differences

Feature	Python	Java
Code blocks	Indentation	Curly braces { }
Typing	Dynamic	Static, typed
for loop	for x in items	Counter or for-each
do-while	Not available	Yes: do { } while
Line end	No semicolon	Semicolon needed

JAVA

```java
for (int i = 1; i <= 3; i++) {
    System.out.println(i);
}
```

PYTHON

```python
for i in range(1, 4):
    print(i)
```

11. Conclusion

Python provides clear syntax for data handling, object-oriented design, calculations, decision-making, and repetition. Classes and attributes help organize related information, while methods and encapsulation support reusable and maintainable code. Operators, conditions, and loops make it possible to build practical applications such as a student management system. Understanding the differences between Python and Java helps programmers transfer concepts between the two languages.